Modulo2_jf

2026-04-15

Introducción

Este es un documento en R Markdown. Permite generar HTML o Word combinando texto y código. El lenguaje R es una poderosa herramienta matemática y estadística. En este reporte generado con R Markdown, analizaremos la resolución de distintos problemas y evaluaremos la calidad de diferentes algoritmos implementados.

Probando primeros comandos de R

R es un leguaje muy parecido a matlab y trabaja con variables que en general pueden ser matrices.

```
A <- 38
A <- 40
B <- 60
C <- B - A
C
```

```
## [1] 20
```

Uso de data sets o base de datos internos de R

Usando el comando *data()* en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan cargados en tablas de las cuales podemos elegir columnas especificas si escribimos el signo *$* despues del comando de informacion especifico que queremos.

También está el comando *summary()*, que es una de las herramientas más útiles y rápidas que tiene R para hacer estadística descriptiva básica.

```
# Planteamos el dataset
data(cars)
```

```
# Usamos el símbolo $ para extraer SOLO la columna de velocidades (speed)
velocidades <- cars$speed

# Mostramos los primeros 6 valores de esa columna aislada
head(velocidades)
```

```
## [1] 4 4 7 7 8 9
```

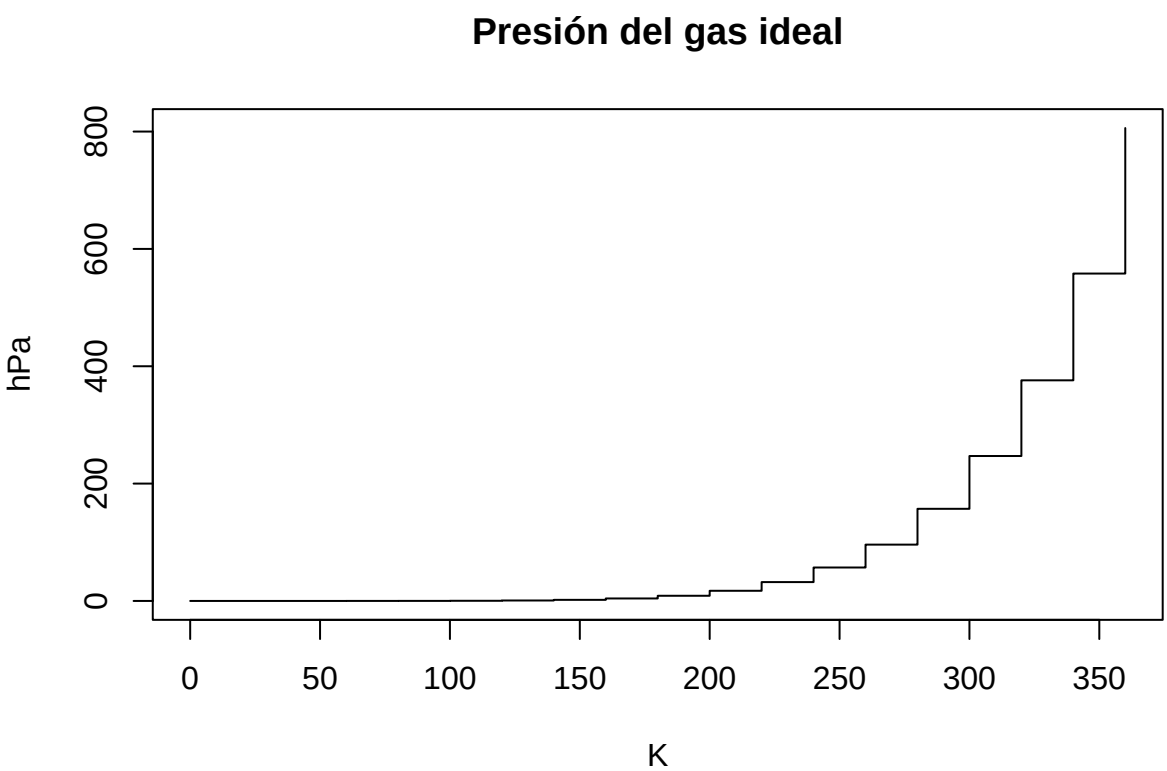
```
summary(cars)
```

```
##      speed           dist
##  Min.   : 4.0   Min.   :  2.00
##  1st Qu.:12.0   1st Qu.: 26.00
##  Median :15.0   Median : 36.00
##  Mean   :15.4   Mean   : 42.98
##  3rd Qu.:19.0   3rd Qu.: 56.00
##  Max.   :25.0   Max.   :120.00
```

Comando plot (Gráficos)

Es un comando que sirve para graficar cualquier fuente de datos que le asignemos: Tenemos la posibilidad de asignar nombres a las variables o hacer cambios del grafico, en caso de no saber el comando podemos escribir **plot** en la consola, a la derecha nos va a abrir una pestaña y ahí tocar Generic X-Y Plotting y ahi nos va a explicar cada comando

```
plot(pressure, type = "s",
     main = "Presión del gas ideal",
     ylab = "hPa",
     xlab = "K")
```



Funciones estadísticas

Otro comando usado es el comando *rnorm*. Este lo que hace es generar datos aleatorios, pero forzando a que esos datos sigan la forma de una Distribución Normal (la famosa campana de Gauss).

Podemos asignarle ciertos parámetros si queremos: *Tamaño de la muestra* **n** (es la cantidad de datos que le pides a R que invente, en este caso 350) *Media* (el valor central, en este caso 22), o *Desviación estándar* (Representa la dispersión o el "error experimental", en este caso, 5).

```
z1 <- rnorm(350, 22, 5)
z1
```

```
##   [1] 22.087988 21.326179 24.187282 22.138531 24.400305 17.049541 22.706035
##   [8] 19.602101 20.345589 23.234423 23.019707 26.192611  8.686592 22.975876
##  [15] 23.226026 15.953727 27.797611 20.183454 22.834866 25.172195 18.612497
##  [22] 18.324077 18.982717 23.532423 24.597788 24.057747 25.204634 31.356601
##  [29] 20.243942 21.970929 24.131386 25.860007 21.682152 26.075912 35.616134
##  [36] 19.321782 32.862956 25.200563 31.348018 24.769526 20.772945 17.885843
##  [43] 24.142550 15.646798 26.132401 22.294711 22.665158 17.930059 26.827331
##  [50] 21.966893 17.967301 14.722153 17.331548 10.787729 24.442481 18.379367
```

```
##  [57] 29.952188 25.077235 33.016864 26.214009 24.005668 23.623373 23.111429
##  [64] 19.996189 31.879069 24.705626 14.205205 25.139894 11.949480 15.001373
##  [71] 19.333973 23.972133 17.635841 14.086449 20.348618 17.741975 27.418376
##  [78] 18.336862 23.328874 21.782912 22.447815 21.197125 14.643031 25.113695
##  [85] 19.734216 18.876365 14.649507 16.014937 22.737608 31.415531 12.233116
##  [92] 23.306885 32.191300 27.461537 20.016294 24.551019 19.288715 23.869285
##  [99] 22.382612 14.237611 18.606000 19.941334 24.237447 26.626538 25.695446
## [106] 21.094397 24.728253 31.468137 24.142164 21.455529 18.199330 21.499436
## [113] 30.706908 22.503256 30.833706 19.022397 20.628448 31.565574 20.520340
## [120] 23.487927 22.865727 16.960322 18.473112 21.026967 27.808999 28.147642
## [127] 21.289922 21.119280 20.186568 15.173363 19.743160 26.496331 19.740762
## [134] 18.429338 12.616719 16.195841 19.266228 14.305290 23.767993 25.300988
## [141] 22.772386 19.125586 27.475343 23.738712 24.764334 27.939347 20.391147
## [148] 19.749257 20.338837 17.371601 20.533879 17.511597 22.359258 26.495444
## [155] 22.090917 17.568749 29.621324 12.927362 26.097155 20.607010 22.531988
## [162] 26.188267 17.142973 23.667231 16.129422 23.174382 22.855523 16.938982
## [169] 20.297093 15.974087 25.837847 15.666618 28.709064 21.935476 32.073641
## [176] 19.938848 17.975966 19.376823 29.159004 17.499762 31.846730 17.465263
## [183] 16.555339 27.015944 23.816724 31.179684 26.226165 18.980032 22.212763
## [190] 26.394786 24.447808 24.772803 22.893109 20.396437 17.309540 28.305884
## [197] 24.810993 38.305497 22.450153 21.984856 17.844670 19.904161 18.106242
## [204] 21.617836 16.920187 26.556578 12.247679 18.927474 19.819841  4.808246
## [211] 23.074670 23.006123 18.872523 27.063032 19.683726 19.137131 17.414657
## [218] 14.619402 27.494336 23.892620 18.434382 15.524586 29.322711 23.259473
## [225] 15.771065 14.731987 16.128493 18.650996 19.766336 19.976189 22.917826
## [232] 19.503114 21.493102 18.027183 24.015367 18.828147 23.128913 23.960349
## [239] 22.338894 27.171617 24.813152 16.910727 32.304307 23.962670 23.479387
## [246] 16.512887 25.276833 15.289962 28.902055 24.514910 13.303923 25.278627
## [253] 19.207125 17.699400 24.648600 18.080884 20.100241 20.625965 21.244984
## [260] 24.373547 33.160295 19.119665 24.215640 19.857773 27.120292 15.322681
## [267] 16.064699 15.942187 23.922956 20.008637 21.854765 31.149595 21.027761
## [274] 16.063807 28.887516 20.587227 16.677365 27.132558 17.934176 24.894460
## [281] 20.157873 27.918474 14.895121 20.391630 26.695353 24.560054 26.012269
## [288] 20.588612 19.775394 28.539543 25.361250 24.123535 15.643281 25.339492
## [295] 25.952825 23.565050 21.271894 23.508187 19.575083 19.568284 23.803268
## [302] 25.418321 20.720312 25.971545 14.555739 23.255415 34.210885 23.876962
## [309] 18.926706 16.430116 16.711401 24.789687 25.673354 16.924008 21.691790
## [316] 25.703471 18.070782 15.901103 28.111901 23.020093 15.746963 32.572464
## [323] 21.113306 21.061505 24.121277 20.870892 21.232487 18.124536 26.165122
## [330] 22.169512 17.777558 20.730474 22.613024 21.164196 23.162308 23.446418
## [337] 25.535177 24.019277 14.282352 23.327693 23.444371 25.156541 28.059012
## [344] 31.188447 18.382846 22.896544 14.673519 21.786976 22.687528 22.569349
```

```r
w1 <- length(z1)
w1
```

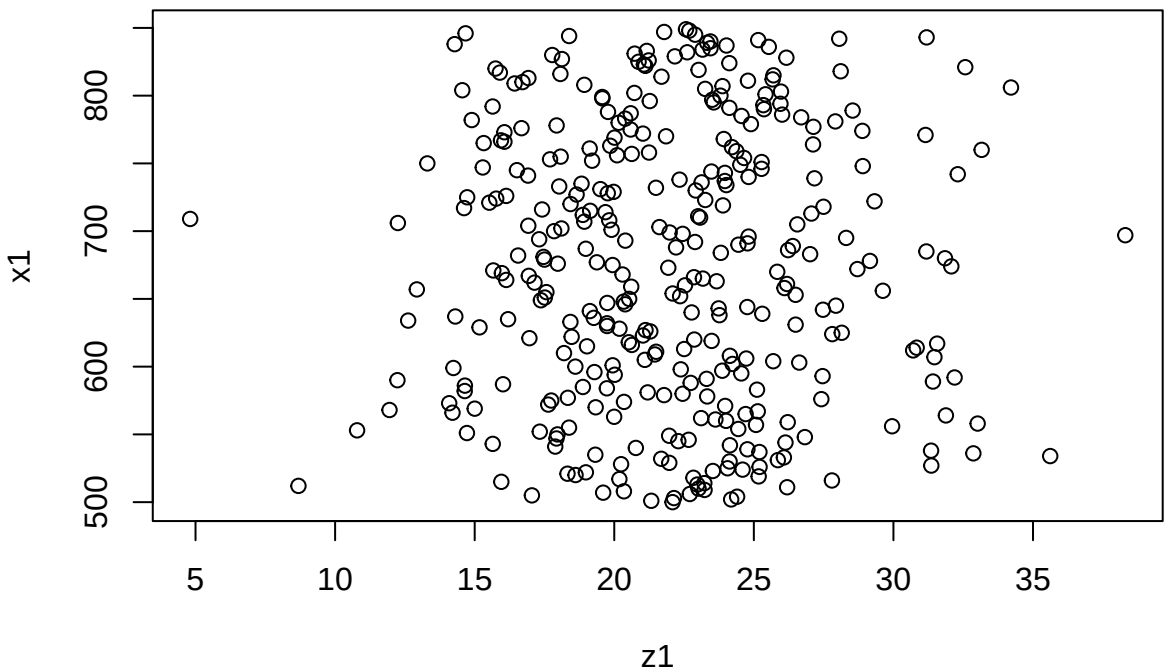
```
## [1] 350
```

```r
x1 <- 500:849
x1
```

```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
```

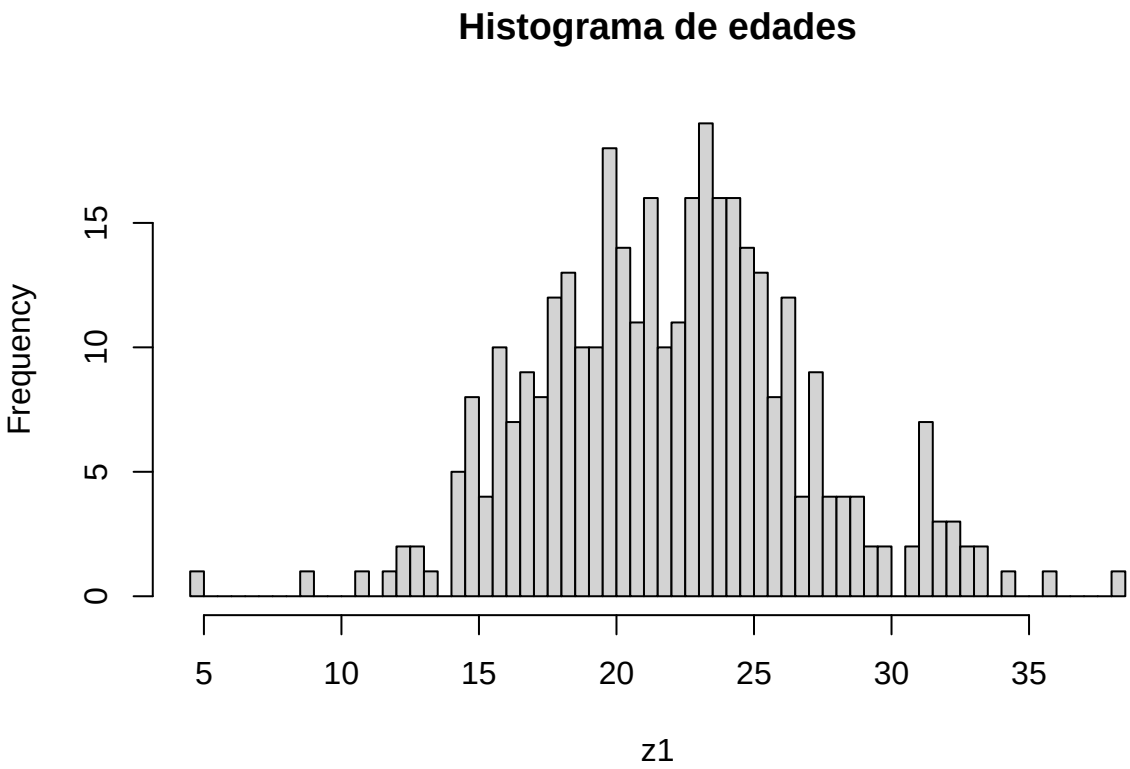
```
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```r
plot(z1, x1)
```



El comando *hist()* genera un histograma.

```r
hist(z1, main = "Histograma de edades", breaks = 60)
```



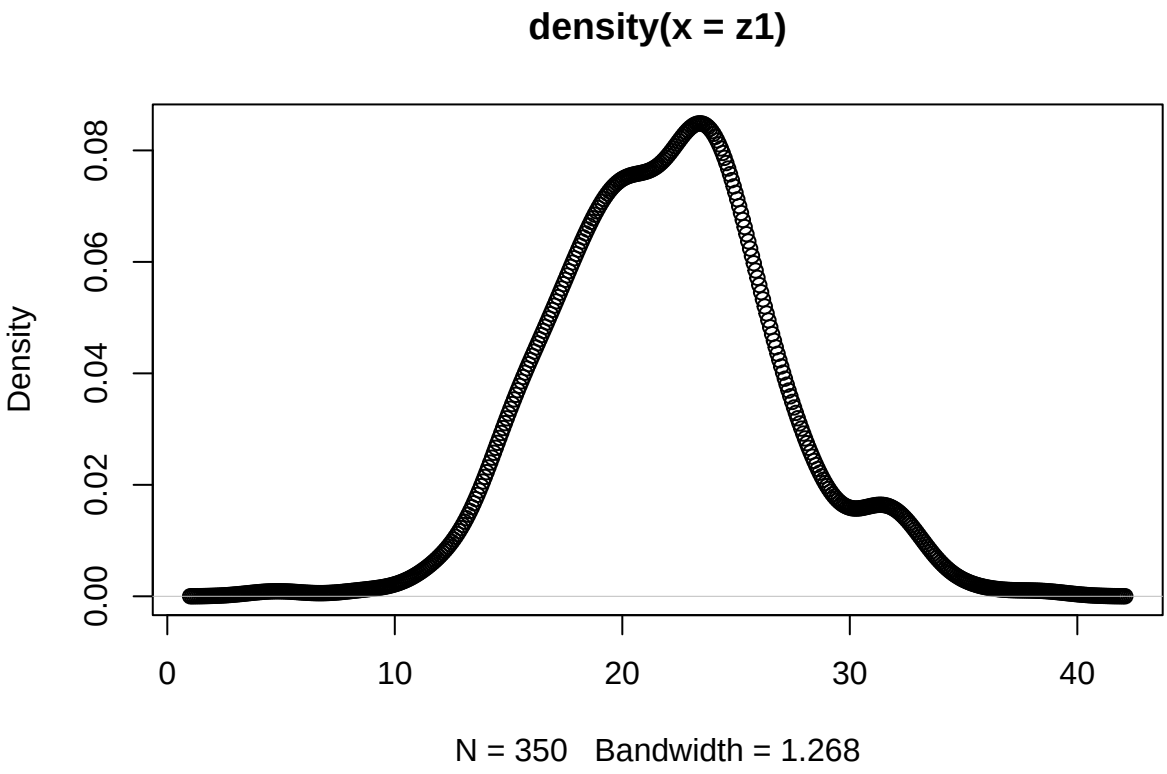
El comando *density()* calcula la densidad de probabilidad. Es la versión matemática, suavizada y continua del histograma.

```
density(z1,type="b")
```

```
## Warning: In density.default(z1, type = "b") :
##  extra argument 'type' will be disregarded
```

```
##
## Call:
##  density.default(x = z1, type = "b")
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.268
##
##        x                y
##  Min.   : 1.006   Min.   :0.0000102
##  1st Qu.:11.281   1st Qu.:0.0009056
##  Median :21.557   Median :0.0101978
##  Mean   :21.557   Mean   :0.0243056
##  3rd Qu.:31.832   3rd Qu.:0.0459272
##  Max.   :42.108   Max.   :0.0848569
```

```
plot(density(z1),type="b")
```



Ejercicio 1: Designación

Consigna: Las ultimas 3 cifras del DNI son 803, crear una variable que tenga ese número

```
DNI <- 803
```

Consigna: Crear un vector del 1 al 803.

```
secuencia_dni <- 1:803
secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
```

```
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450
## [451] 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468
## [469] 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486
## [487] 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504
## [505] 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522
## [523] 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540
## [541] 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558
## [559] 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576
## [577] 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594
## [595] 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612
## [613] 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630
## [631] 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648
## [649] 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666
## [667] 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684
## [685] 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702
## [703] 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720
## [721] 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738
## [739] 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756
## [757] 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774
## [775] 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792
## [793] 793 794 795 796 797 798 799 800 801 802 803
```

Ejercicio 2: Uso de for

Consigna: Calcular la suma de todos los valores del vector *secuencia_dni* usando un for:

```r
Total <- 0
valor_final <- length(secuencia_dni)

for (i in 1:valor_final) {
  Total <- Total + i
}

Total
```

```
## [1] 322806
```

Ejercicio 3: Phyton

Consigna: Repetir el ejercicio anterior pero en Phyton

```python
# --- Ejercicio 1 ---
# Consigna: Crear una variable con las últimas 3 cifras del DNI (803)
dni = 803

# Consigna: Crear un vector (lista) del 1 al 803
# En Python, range(1, 804) genera números del 1 al 803
```

```python
secuencia_dni = list(range(1, dni + 1))

# --- Ejercicio 2 ---
# Calcular la suma de todos los valores usando un bucle 'for'
total = 0

for i in secuencia_dni:
    total += i

# Mostrar el resultado final
print(f"La suma total es: {total}")
```

```
## La suma total es: 322806
```

Ejercicio 4: Medición de tiempo (sys.time)

Consigna:¿Cuanto tarda en correr el código del ejercicio 2?

Usamos el comando **sys.time**

```r
inicio <- Sys.time()
total <- 0
valor_final <- 10000000*length(secuencia_dni)
for (i in 1:valor_final)
total <- total + i
total
```

```
## [1] 3.224045e+19
```

```r
final <- Sys.time()
final-inicio
```

```
## Time difference of 2.243216 mins
```

Ejercicio 5: 2da medición de tiempo (tictoc)

Consigna: Aplicar la consigna anterior pero aprendiendo a usar el comando **tictoc**

```r
library(tictoc)

# Iniciamos el cronómetro asignándole un nombre descriptivo a la prueba
tic("Tiempo de ejecución")

# Inicio del bloque a medir
Total<-0
valor_final<-length(secuencia_dni)

for(i in 1:valor_final) {
Total<-Total + i
}
# Fin del bloque a medir

#Se "detiene" el cronómetro.
```

```r
#Esto imprimirá el tiempo transcurrido en pantalla.
toc()
```

```
## Tiempo de ejecución: 0.005 sec elapsed
```

```r
#Imprimimos la variable Total para verificar que el cálculo matemático se hizo bien
Total
```

```
## [1] 322806
```

Ejercicio 6: Secuencia

Consigna: Generar secuencia de 2 en 2 hasta 50000

```r
inicio_for <- Sys.time()

A <- numeric(50000)
for (i in 1:50000) {
  A[i] <- i * 2
}

final_for <- Sys.time()
tiempo_for <- final_for - inicio_for
```

Secuencia con función de R:

```r
inicio_seq <- Sys.time()

B <- seq(2, 100000, by = 2)

final_seq <- Sys.time()
tiempo_seq <- final_seq - inicio_seq

# Resultados
tiempo_for
```

```
## Time difference of 0.00625515 secs
```

```r
tiempo_seq
```

```
## Time difference of 0.002112627 secs
```

Ejercicio 7: Serie de Fibonacci

Consigna: Generar sucesión o serie de Fibonacci hasta superar 1.000.000

```r
# Generar Fibonacci hasta superar 1.000.000

fibonacci <- c(0,1)
iteraciones <- 2

while (fibonacci[length(fibonacci)] <= 1000000) {
  nuevo <- fibonacci[length(fibonacci)] + fibonacci[length(fibonacci)-1]
  fibonacci <- c(fibonacci, nuevo)
  iteraciones <- iteraciones + 1
```

```
}

fibonacci
```

```
##  [1]       0       1       1       2       3       5       8      13      21
## [10]      34      55      89     144     233     377     610     987    1597
## [19]    2584    4181    6765   10946   17711   28657   46368   75025  121393
## [28]  196418  317811  514229  832040 1346269
```

```
iteraciones
```

```
## [1] 32
```

Ejercicio 8: Definición matemática recurrente

Consigna:¿Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000?

```
f0<-0   #Valores iniciales
f1<-1
f2<-0
i<-0  #Número de iteraciones incial
valorfinal<-1000000 #Umbral

while (f2<valorfinal) {

  f2=f0+f1
  f0=f1
  f1=f2
  i<-i+1

}
f2
```

```
## [1] 1346269
```

```
i
```

```
## [1] 30
```

Ejercicio 9: Método Burbuja (Bubble Sort)

Consigna: Compara la performance de ordenación del método burbuja vs el método sort de R.

```
x <- sample(1:20000, 20000)

burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n-1)) {
    for (i in 1:(n-j)) {
      if (x[i] > x[i+1]) {
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
```

```r
  }
  return(x)
}
```

Comparación:

```r
system.time({
  res1 <- burbuja(x)
})
```

```
##    user  system elapsed
##  21.996   0.008  22.135
```

```r
system.time({
  res2 <- sort(x)
})
```

```
##    user  system elapsed
##   0.001   0.000   0.001
```

Ejercicio 10: Penitencia de Newton

Consigna: Desarrollar dos algoritmos que hagan el trabajo de sumar desde 1 hasta 1000000 y verifcar cuál de los dos es más eficiente

Método 1 (for):

```r
n <- 1000000

system.time({
  suma1 <- 0
  for (i in 1:n) {
    suma1 <- suma1 + i
  }
})
```

```
##    user  system elapsed
##   0.017   0.000   0.017
```

Método 2 (fórmula matemática):

```r
system.time({
  suma2 <- n * (n + 1) / 2
})
```

```
##    user  system elapsed
##   0.000   0.000   0.001
```

Comparación:

```r
suma1
```

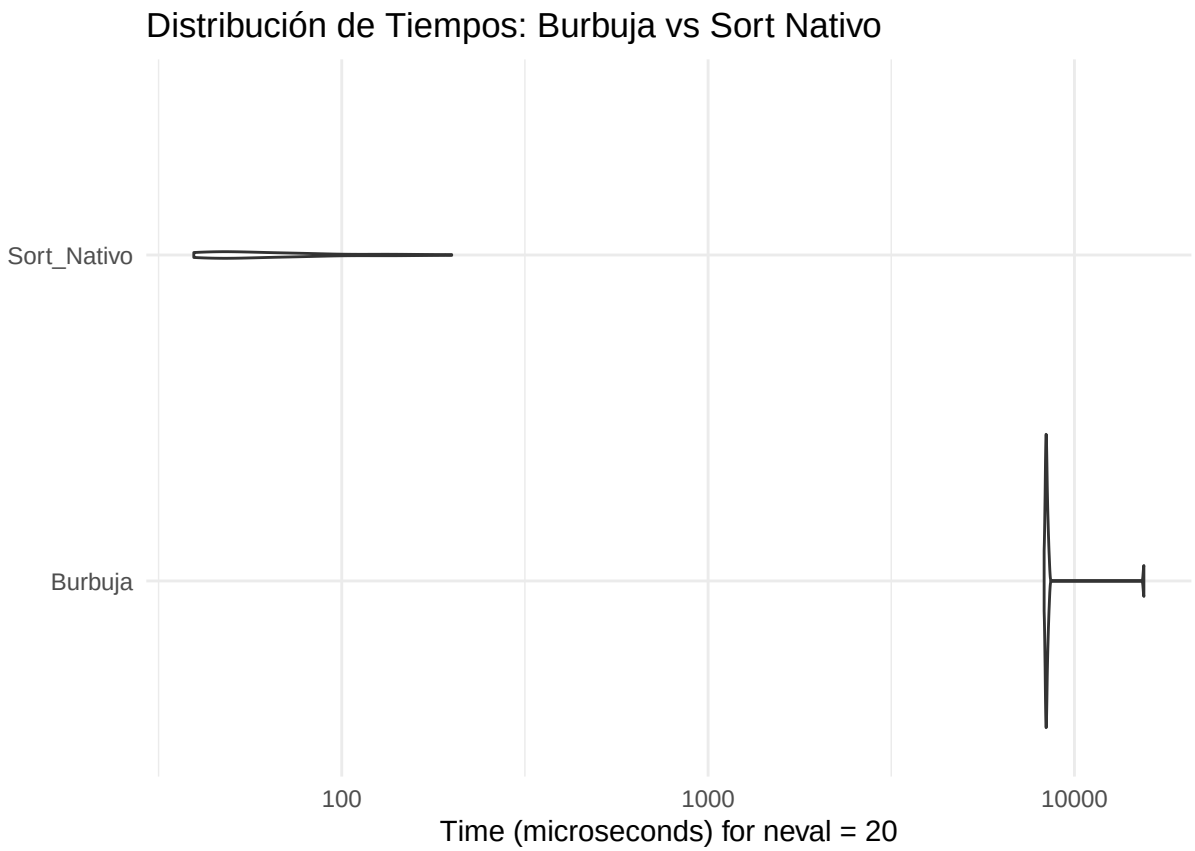
```
## [1] 500000500000
```

```r
suma2
```

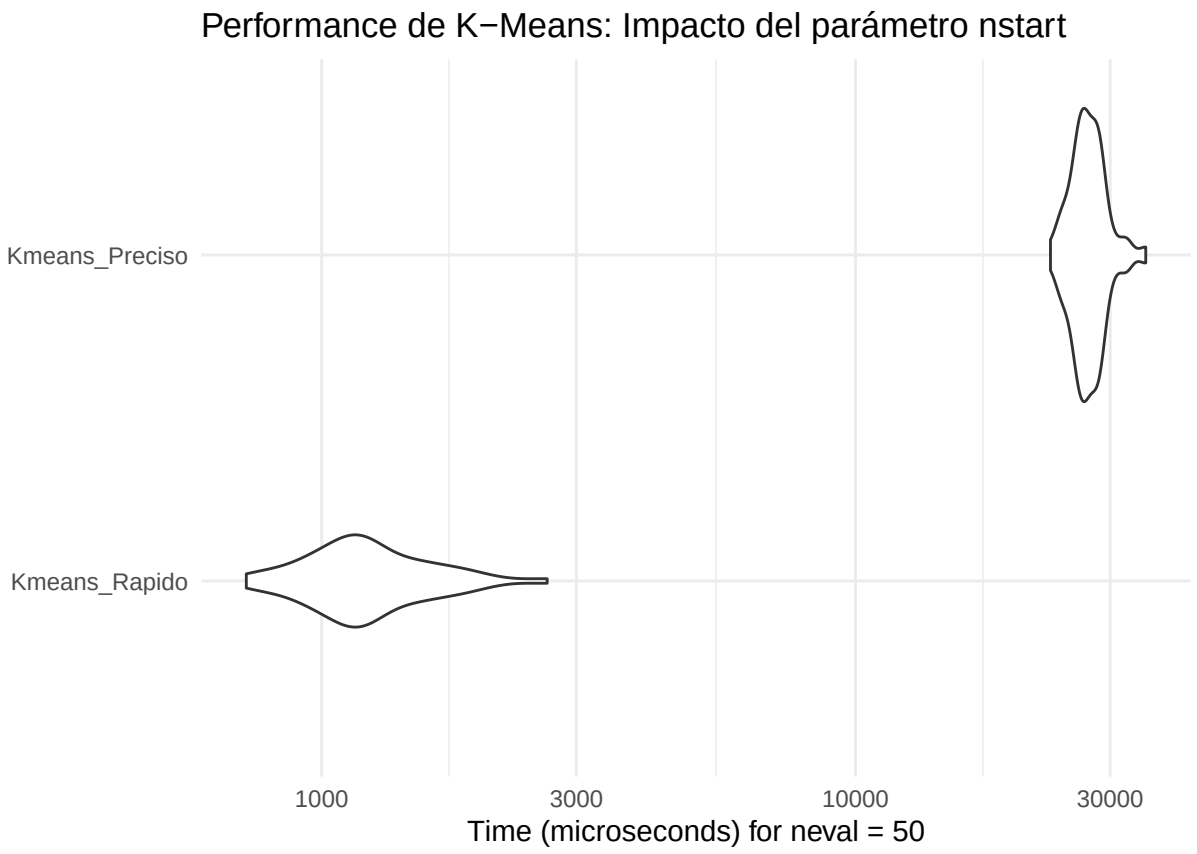
```
## [1] 500000500000
```

Análisis de Rendimiento: Ordenamiento y Agrupación

```r
library(microbenchmark)
library(ggplot2)

# Definimos la función del método burbuja (del apunte original)
burbuja <- function(x){
  n <- length(x)
  for(j in 1:(n-1)) {
    for(i in 1:(n-j)){
      if(x[i] > x[i+1]){
        temp <- x[i]
        x[i] <- x[i+1]
        x[i+1] <- temp
      }
    }
  }
  return(x)
}

# Generamos una muestra aleatoria moderada para la prueba
set.seed(123)
muestra <- sample(1:1000, 400, replace = TRUE)

# Ejecutamos el benchmarking
mbm_orden <- microbenchmark(
  Burbuja = burbuja(muestra),
  Sort_Nativo = sort(muestra),
  times = 20 # Repetimos 20 veces para tener una buena distribución
)

# Visualizamos con gráfico de violín
autoplot(mbm_orden) +
  ggtitle("Distribución de Tiempos: Burbuja vs Sort Nativo") +
  theme_minimal()
```




```r
# Generamos una matriz de datos simulada (ej: variables de mantenimiento industrial)
set.seed(42)
datos_industriales <- matrix(rnorm(5000), ncol = 5)

# Benchmarking del algoritmo K-Means
mbm_kmeans <- microbenchmark(
  Kmeans_Rapido = kmeans(datos_industriales, centers = 4, nstart = 1),
  Kmeans_Preciso = kmeans(datos_industriales, centers = 4, nstart = 25),
  times = 50
)

# Visualizamos con gráfico de violín
autoplot(mbm_kmeans) +
  ggtitle("Performance de K-Means: Impacto del parámetro nstart") +
  theme_minimal()
```



Conclusión final

A lo largo de este trabajo, evaluamos la eficiencia de distintos algoritmos en R mediante técnicas de benchmarking (como Sys.time, tictoc y microbenchmark). Comprobamos empíricamente que la calidad de un código no radica solo en llegar al resultado correcto, sino en cómo optimiza el tiempo y los recursos del sistema.

La incorporación de los gráficos de violín resultó fundamental para visualizar estadísticamente no solo la media de ejecución, sino la estabilidad y distribución temporal de procesos analíticos complejos, como la comparación de métodos de ordenamiento y la medición de performance del agrupamiento K-Means. Es fundamental destacar que los tiempos absolutos medidos y la forma exacta de la "panza" de los violines serán diferentes en cada computadora; estos resultados variarán dinámicamente según las propiedades físicas y la capacidad de procesamiento (hardware) del equipo donde se compile el documento.

Sin embargo, más allá de la máquina específica que se utilice, el principio de optimización estructural se mantiene inalterable. El ejercicio de "La penitencia de Newton" ilustró perfectamente este concepto: mientras la fuerza bruta de un bucle iterativo demanda un costo computacional inmenso para realizar operaciones masivas, la aplicación de un modelo matemático sólido resuelve el problema en fracciones de segundo.

En definitiva, la evaluación de estos algoritmos nos confirma que la herramienta de optimización más poderosa en la programación moderna no es la computadora que tengamos, sino el razonamiento lógico, físico y matemático que aplicamos antes de escribir la primera línea de código.